

Lab 06 - Local Features: Colour Segmentation, Canny Edges and Corner Keypoints

Course: Generative AI for Image and Video Creation (TGS-2020505925) | Version v11.0 | 6 September 2026 Topic 03: AI-Based Visual Features, Styles and Consistency Outcome: ELO3 | In-class time: 30 minutes of scheduled practical time

Competency	Statement
Ability A4	Design and implement feature extraction and representation methods
Knowledge K5	Feature extraction and representation techniques
Knowledge K6	Local feature descriptions, edge, colour, texture and motion

Scenario

The campaign QC bot has to answer three questions about every delivered asset: is the brand colour present and in the right place, is the product silhouette crisp, and can this frame be matched to the approved master. Those are colour, edge and keypoint questions. Implement all three and record the transformations each tolerates and its failure cases.

What you will produce

out/features-report.json with the HSV segmentation coverage, the Canny threshold sweep, response/count measurements for five detectors, and an ORB match count between the asset and its rotated copy.

Tools: Python 3 | opencv-python | NumPy | no network, no paid service

Environment

Install these once, before class if you can - the lab itself needs no network access after the dependencies are present:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2, numpy; print(cv2.__version__, numpy.__version__)"
```

Required: python3, opencv-python, numpy.

API currency. Everything taught here runs on the base opencv-python wheel. Two things that appear in older tutorials are **not** in that wheel and are deliberately avoided: cv2.TrackerCSRT_create / cv2.TrackerKCF_create and the cv2.quality SSIM module, both of which live in opencv-contrib-python. Where SSIM is needed the course ships its own NumPy implementation.

Workflow

1. Segment the brand colour in HSV
2. Sweep Canny thresholds and pick a ratio
3. Detect Harris and Shi-Tomasi corners

4. Detect FAST, ORB and SIFT keypoints
5. Match a rotated copy with ORB

Files in this folder

Path	What it is
reference/nq-asset-master.png	Synthetic 900x700 campaign asset containing the slate-blue brand block, a hard-edged product silhouette, a fine-texture panel and a chequered calibration corner (deterministically rendered with OpenCV - SIMULATED)
reference/nq-asset-variant.png	The same asset rotated 20 degrees and scaled to 0.8, for the matching step (deterministically rendered - SIMULATED)
data/brand-colour.json	The brand hue band, the required coverage range and the required location quadrant
data/expected-features.json	Reference keypoint counts and match counts for self-verification (tolerances included, since detector counts vary slightly across OpenCV builds)
features.py	Runnable script - see the steps below
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack
out/	Everything you produce (created on first run)

Provenance. Every image and clip in reference/ is either a SIMULATED classroom asset rendered deterministically with OpenCV, an AI-generated reference copied unchanged with a PROVENANCE.md beside it, or a real prerecorded sequence with its source and SHA-256 recorded. Nothing in this folder may be relabelled: a rendered animation is never presented as model output, and a simulated stand-in is never presented as a photograph.

Steps

1. Load reference/nq-asset-master.png and convert it to HSV. Remember the OpenCV 8-bit convention from Lab 02: H is 0-179, S and V are 0-255.

```
img = cv2.imread('reference/nq-asset-master.png')
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

2. Read data/brand-colour.json. The slate-blue brand hue band is given as an H range with S and V floors. Build the mask with cv2.inRange and report coverage as a percentage of the frame.

```
mask = cv2.inRange(hsv, tuple(spec['lo']), tuple(spec['hi']))
print('brand coverage %.2f%%' % (100.0 * (mask > 0).mean()))
```

3. Test exposure robustness on this fixture, not a universal invariance claim. Multiply the V channel by 0.6 to simulate an under-exposed delivery, re-run the same inRange, and compare coverage. Then do the same experiment thresholding the BLUE channel of the BGR image instead. Record both coverage changes. Hue separation helps here, but V floors, clipping, noise and white-balance shifts can still break segmentation.

```
dark = hsv.copy(); dark[:, :, 2] = (dark[:, :, 2] * 0.6).astype('uint8')
print('HSV coverage under -40%% V: %.2f%%' % (100.0 * (cv2.inRange(dark, lo, hi) > 0).mean()))
```

4. Handle the red wrap-around case explicitly, even though this asset is blue. Red spans H near 179 and H near 0, so it needs two inRange calls combined with cv2.bitwise_or. Write the two-band code and keep it - every HSV

pipeline meets red eventually.

```
m1 = cv2.inRange(hsv, (0, 120, 80), (8, 255, 255))
m2 = cv2.inRange(hsv, (172, 120, 80), (179, 255, 255))
red = cv2.bitwise_or(m1, m2)
```

5. Check the LOCATION requirement from data/brand-colour.json: the brand block must sit in the stated quadrant. Compute the mask centroid with cv2.moments and test which quadrant it falls in. Coverage alone does not prove correct placement.
6. Move to edges. Smooth grayscale with cv2.GaussianBlur(gray, (5,5), 1.0) first, then run cv2.Canny at (50,150), (100,200) and (150,300) and record the edge-pixel density of each. Canny takes two thresholds because it uses hysteresis: pixels above the high threshold seed strong edges; weaker pixels above the low threshold that connect to strong edges continue one.
7. Name the four Canny stages in your report - Gaussian smoothing, Sobel gradient magnitude and direction, non-maximum suppression, hysteresis thresholding. Canny is an algorithm, not a single convolution; you cannot tune it sensibly without knowing which stage each parameter touches.
8. Sweep the low:high ratio at a fixed high threshold of 200: try low = 200, 100, 66 (ratios 1:1, 1:2, 1:3). Record the edge density for each and pick the ratio that keeps the product silhouette continuous without lighting up the texture panel.

```
python3 features.py --image reference/nq-asset-master.png --brand data/brand-colour.json --
out out/features-report.json
```

9. Now corners (the Harris count is response pixels, not distinct corners). Run cv2.cornerHarris(gray32, blockSize=2, ksize=3, k=0.04), dilate the response, and count the pixels above 1% of the maximum response. Harris measures intensity variation in ALL directions - that is what distinguishes a corner from an edge.

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
dst = cv2.cornerHarris(np.float32(gray), 2, 3, 0.04)
dst = cv2.dilate(dst, None)
print('harris response pixels', int((dst > 0.01 * dst.max()).sum()))
```

10. Run cv2.goodFeaturesToTrack (Shi-Tomasi) with maxCorners=200, qualityLevel=0.01, minDistance=10. Shi-Tomasi uses the minimum eigenvalue of the structure tensor with an explicit spacing constraint, which returns a separated point set rather than a response map.
11. Rotate the image 30 degrees with cv2.getRotationMatrix2D + cv2.warpAffine, re-run Harris, and compare the count. Then SCALE the image to 0.5 and re-run. Record both. Harris is rotation-covariant in theory, but interpolation, cropping and thresholding change counts. Fixed-window Harris is not scale-invariant; count equality alone does not measure repeatability of corresponding corners.
12. Run cv2.FastFeatureDetector_create(). FAST tests a ring of 16 pixels around a candidate and accepts it if a contiguous arc is all brighter or all darker than the centre by a threshold. It is a detector only - it produces no descriptor, so it cannot match on its own.
13. Run cv2.ORB_create(nfeatures=500) with detectAndCompute. ORB gives you both keypoints and a 32-byte binary descriptor, and it is free of patent restrictions. Print descriptors.shape and confirm the second dimension is 32.
14. Run cv2.SIFT_create() with detectAndCompute and print descriptors.shape. The second dimension is 128 floats. Compare the descriptor sizes: 32 bytes versus 512 bytes per keypoint is a 16x storage and bandwidth difference at the same keypoint count - record it, because Topic 5 turns it into a cost number.
15. Match the master against reference/nq-asset-variant.png (rotated 20 degrees, scaled 0.8) using ORB plus cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True). Hamming distance is the correct metric for a binary descriptor; L2 is not.

```
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = sorted(bf.match(des1, des2), key=lambda m: m.distance)
print('matches', len(matches), 'best distance', matches[0].distance)
```

16. Apply a median-distance heuristic: keep matches at or below 0.75 x the median distance. This is NOT Lowe's nearest/second-nearest ratio test. Report the surviving count and inspect correspondence geometry; neither filter proves identity.

17. Compare every count against data/expected-features.json, which carries tolerances because keypoint counts shift slightly between OpenCV builds. Save out/features-report.json and complete the evidence checklist.

Verify

out/features-report.json contains brand coverage before and after the exposure change (explain the observed difference and the V-floor limitation), edge densities for three Canny settings, keypoint counts for Harris, Shi-Tomasi, FAST, ORB and SIFT, the Harris rotation and scale comparison, and a filtered ORB match count above zero between the master and the rotated variant.

Expected outputs

- Brand-colour coverage within the range stated in data/brand-colour.json.
- Measure both coverage changes; the synthetic V scaling preserves H and S by construction, but pixels crossing the V floor can still leave the mask.
- Edge density is nondecreasing as thresholds fall on the same smoothed image; inspect which ratio keeps the silhouette continuous.
- Report Harris response-pixel counts under rotation and scaling without treating them as matched corner counts.
- ORB descriptor width 32; SIFT descriptor width 128.
- The supplied OpenCV 4.13 fixture produces 15 filtered ORB matches (79 raw); allow build variation, and inspect geometry. The illustrative gate minimum is 10.

Evidence checklist

Submit all of the following. Each item is something an assessor can look at.

- [] out/features-report.json attached.
- [] The two coverage figures for the exposure test (HSV vs raw blue channel).
- [] The centroid quadrant of the brand mask and whether it met the placement rule.
- [] The four Canny stages named, and the low:high ratio you selected with a reason.
- [] Harris counts at 0 degrees, 30 degrees and 0.5 scale.
- [] ORB and SIFT descriptor widths, and the 16x storage comparison.
- [] The raw and median-heuristic-filtered ORB match counts and an inspected correspondence visual.

Troubleshooting

Symptom	What to do
cv2.cornerHarris raises an assertion	It requires a single-channel float32 input. Convert with <code>np.float32(cv2.cvtColor(img, cv2.COLOR_BGR2GRAY))</code> .

Symptom	What to do
SIFT is not available	SIFT has been in the main OpenCV distribution since 4.4 - if <code>cv2.SIFT_create</code> is missing you are on an old wheel. <code>pip install --upgrade opencv-python</code> . Do NOT substitute SURF: it remains non-free and is not in the standard package.
<code>bf.match</code> raises an error about descriptor type	You mixed a binary and a float descriptor, or passed <code>NORM_HAMMING</code> to SIFT. Use <code>NORM_HAMMING</code> for ORB/BRIEF/BRISK and <code>NORM_L2</code> for SIFT.
<code>detectAndCompute</code> returns <code>None</code> for descriptors	No keypoints were found - usually a blank or uniformly flat crop. Check the image loaded, and lower the detector threshold.
Brand coverage is 0%	Your hue band is in degrees rather than OpenCV's halved scale. A 210-degree slate blue is <code>H=105</code> in OpenCV, not 210.
Match count collapses to near zero after filtering	<code>crossCheck=True</code> plus a 0.75x-median filter is deliberately strict. Report the number; the count alone does not show which matches are geometrically correct. Zero-distance matches survive the inclusive cutoff even when the median is zero.

References used in this lab

- [S17] Mewada et al. (2026), as above - *Ch.7 pp.94-98 (Dubey, Pinheiro, Singh, Ansari and Kumar, 'Advanced Foundations and Future Trends in Generative AI for Visual Media')*.

Table 7.2 scaling strategies and their targets (trajectory distillation -> fewer sampling steps at similar FID/FVD; quantisation 8/4-bit -> reduced latency and memory; low-rank adapters -> small trainable-parameter fraction; latent diffusion -> lower bandwidth cost at high resolution; retrieval conditioning -> higher faithfulness without more parameters). Evaluation protocol: FID as a 2-Wasserstein approximation in feature space (encoder-dependent, sample-size sensitive), KID as an unbiased MMD^2 , LPIPS as a learned perceptual distance, precision/recall for mode coverage, CLIP-based text-image faithfulness, masked-region consistency and identity preservation for edits, FVD plus optical-flow agreement for video, and watermark detectability under common edits (Table 7.3 axes and pitfalls).

- [S23] OpenCV 4.13 Python package, verified locally on the delivery image - *cv2 4.13.0 (opencv-python 4.13.0.92)*.

API-currency verification for every command taught: `cv2.data.harcascades` ships the frontal-face and eye cascades so no weights download is required; `cv2.HOGDescriptor_getDefaultPeopleDetector()` is built in; `SIFT_create`, `ORB_create`, `FastFeatureDetector_create`, `cornerHarris`, `goodFeaturesToTrack`, `matchTemplate`, `createBackgroundSubtractorMOG2/KNN`, `meanShift`, `CamShift`, `calcOpticalFlowFarneback` and `PSNR` are all present. IMPORTANT CORRECTION carried into the labs: `cv2.TrackerCSRT_create` and `cv2.TrackerKCF_create` are NOT in the base `opencv-python` wheel (they require `opencv-contrib-python`), and the `cv2.quality SSIM` module is also contrib-only - so the labs use `cv2.TrackerMIL_create` and a NumPy SSIM implementation to stay dependency-light and runnable offline.

- [S14] A. Mewada, M. A. Ansari, S. Ahmad and N. Singh (eds), 'AI-Generated Image and Video Synthesis', IEEE Press / Wiley, 2026, ISBN 9781394403110 - *Ch.1 pp.1-11*.

Historical evolution of synthetic media; role of data in training generative models; overview of major architectures; text-to-image, style transfer and super-resolution as distinct image-synthesis tasks; frame-based versus temporal video generation, motion transfer, and the temporal-consistency and realism challenges.

© 2026 Tertiary Infotech Academy Pte Ltd. UEN: 201200696W. Course material for TGS-2020505925, version v11.0.